



شبکه‌های کامپیوتری

Amirhossein Mansouri & Alireza karimi



۸۱۰۱۰۲۳۴۳

۸۱۰۱۰۱۴۹۲

NS2 CA

phase1

چکیده

هدف از این پروژه، شبیه‌سازی و ارزیابی عملکرد سه نسخه اصلی پروتکل TCP شامل Reno، Tahoe و Vegas در شرایط محدودیت پهنای باند است. با استفاده از شبیه‌ساز شبکه NS2، یک توپولوژی دمبلی (Dumbbell) پیاده‌سازی شد که در آن یک لینک گلوگاه (Bottleneck) با پهنای باند بسیار کم (100kbps) وجود دارد. نتایج حاصل از ۳۰ بار اجرای شبیه‌سازی (۱۰ اجرا برای هر پروتکل) نشان می‌دهد که هر سه پروتکل موفق به اشتراک‌گذاری عادلانه پهنای باند شدند، اما TCP Vegas به دلیل مکانیزم پیشگیرانه خود، نرخ از دست رفتن بسته (Packet Loss) کمتری نسبت به دو پروتکل دیگر داشت. همچنین به دلیل صف‌بندی طولانی در گلوگاه، هر سه پروتکل تاخیر رفت و برگشت (RTT) بالایی را تجربه کردند.

۱ سناریوی آزمایش

ما شبکه‌ای را طراحی کردیم که ترافیک سنگینی را به سمت یک لینک ضعیف هدایت می‌کند تا رفتار پروتکل‌ها را در شرایط ازدحام (Congestion) بررسی کنیم. مشخصات توپولوژی به شرح زیر است:

۱. فرستنده‌ها: گره ۱ (Flow 1) و گره ۲ (Flow 2)

۲. گیرنده‌ها: گره ۵ و گره ۶

۳. لینک گلوگاه (از Router 1 به Router 2):

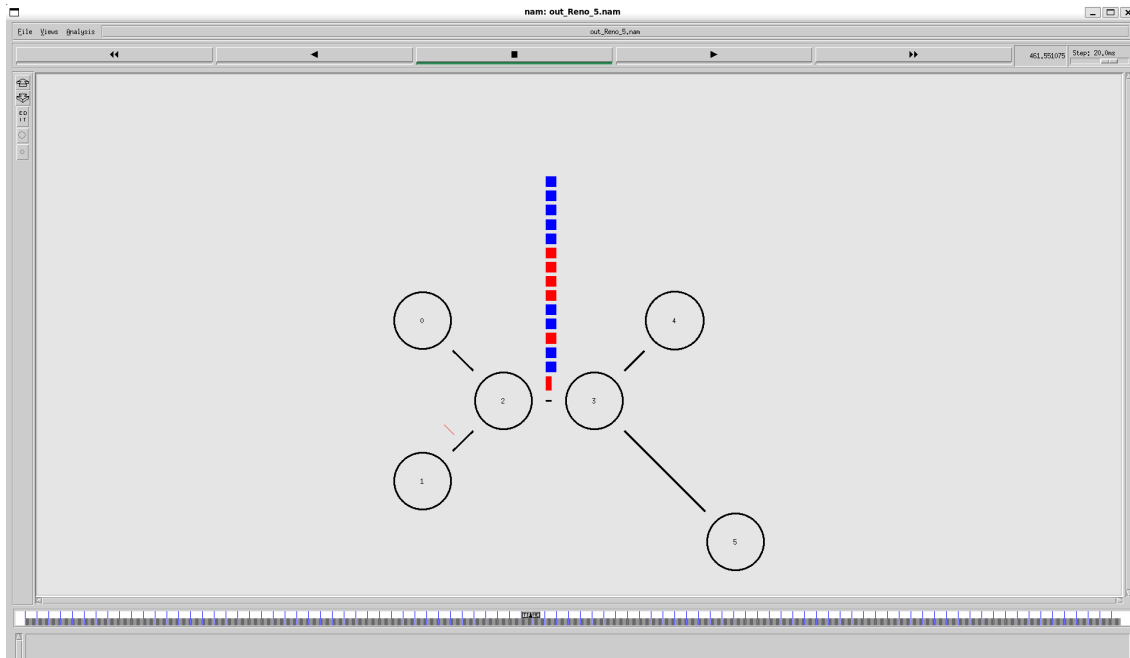
(آ) پهنای باند: 100 kbps (بسیار محدود)

(ب) تاخیر: 1 ms

(ج) صف (Queue): نوع DropTail با ظرفیت ۱۰ بسته

۴. لینک‌های دسترسی: پهنای باند 100 Mbps (پرسرعت)

۵. متغیرهای تصادفی: برای واقعی‌تر شدن نتایج، تاخیر لینک‌های فرستنده در هر اجرا بین ۵ تا ۲۵ میلی‌ثانیه متغیر است و زمان شروع جریان‌ها دارای Jitter (لرزش زمانی) تصادفی است



شکل ۱: توپولوژی دمبلی شبکه

در این بخش، منطق کدهای نوشته شده را خط به خط بررسی می‌کنیم.

۲ موتور شبیه‌سازی (project_sim.tcl)

این فایل مغز متفکر شبیه‌سازی است که توپولوژی و منطق شبکه را تعریف می‌کند.

۱.۲ ایجاد تصادفی‌سازی پیشرفته

ما از تولیدکننده اعداد تصادفی (RNG) مستقل استفاده کردیم:

```
set rng_delay [new RNG]; $rng_delay seed $seed

set rand_delay [new RandomVariable/Uniform]
$rand_delay set min_ 5.0; $rand_delay set max_ 25.0
```

تحلیل: برخلاف شبیه‌سازی‌های ساده که از اعداد صحیح استفاده می‌کنند، ما از اعداد اعشاری (Floating-point) استفاده کردیم تا تاخیرها دقیقاً یکسان نباشند. این کار باعث می‌شود هر اجرا (Run) منحصر به فرد باشد.

۲.۲ تعریف گلوگاه

```
$ns duplex-link $n2 $n3 100Kb 1ms DropTail
$ns queue-limit $n2 $n3 10
```

تحلیل: این خط مهم‌ترین بخش پروژه است. پهنای باند ۱۰۰ کیلوبیت بسیار کم است. با توجه به اینکه هر بسته ۱۰۰۰ بایت (۸۰۰۰ بیت) است، لینک فقط می‌تواند حدود ۱۲ بسته در ثانیه عبور دهد. محدودیت صف ۱۰ تایی باعث می‌شود بسته‌ها به سرعت در صف انباشته شده و دور ریخته شوند (Drop).

۳.۲ مانیتورینگ RTT

از آنجا که فایل‌های Trace استاندارد، RTT را مستقیماً نشان نمی‌دهند، ما یک تابع اختصاصی نوشتیم:

```
proc record_rtt {} {
    set rtt1 [$tcp1 set rtt_]
    puts $rttfile "$now $rtt1 ..."
    $ns at [expr $now + 0.5] "record_rtt"
}
```

تحلیل: این تابع هر ۰.۵ ثانیه مقدار متغیر داخلی `rtt_` را از ایجنت TCP می‌خواند و در یک فایل جداگانه ذخیره می‌کند تا بتوانیم نمودار تغییرات تاخیر در زمان را رسم کنیم.

۳ مدیریت خودکار (run_project.sh)

این اسکریپت وظیفه اجرای تکراری آزمایش‌ها و میانگین‌گیری را بر عهده دارد:

۱. **حلقه اجرا:** کد شبیه‌سازی را ۳۰ بار (۱۰ بار برای هر پروتکل) اجرا می‌کند

۲. **محاسبات AWK:**

(آ) گذردهی (Throughput): مجموع بایت‌های دریافتی ضرب در ۸، تقسیم بر زمان

(ب) نرخ اتلاف (Packet Loss): (تعداد بسته‌های ارسالی - دریافتی) تقسیم بر ارسالی

۳. **ذخیره‌سازی:** تمام نتایج خام در فایل `results/all_runs_raw.csv` ذخیره می‌شوند تا قابلیت رسم نمودارهای دقیق با نوار خطا (Error Bars) فراهم شود

در این بخش عملکرد سه پروتکل را بر اساس نمودارهای استخراج شده مقایسه می‌کنیم.

۴ مقایسه گذردهی

۱.۴ مشاهدات

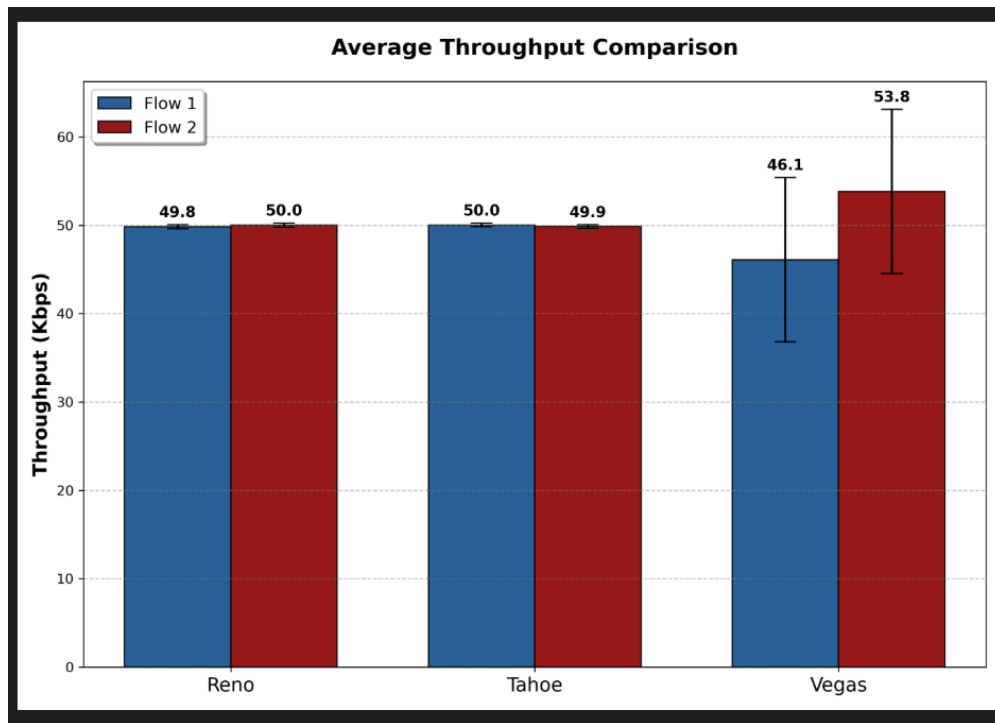
۱. در تمامی پروتکل‌ها، مجموع گذردهی دو جریان به حدود 100 kbps (ظرفیت لینک گلوگاه) می‌رسد

۲. هر جریان به طور متوسط حدود 50 kbps سهم می‌برد

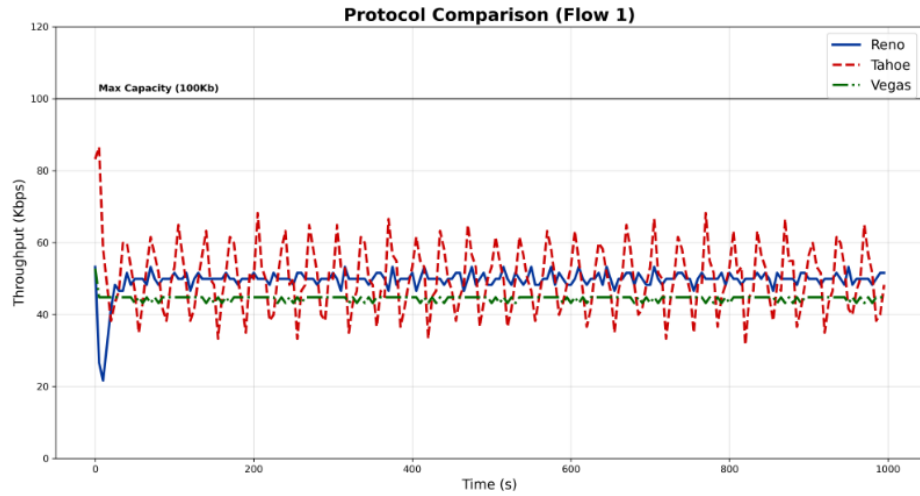
۲.۴ تحلیل عمیق

این نتیجه نشان‌دهنده عدالت (Fairness) در پروتکل TCP است. با وجود اینکه شرایط لینک‌ها در هر اجرا متفاوت بود، الگوریتم‌های کنترل ازدحام به گونه‌ای طراحی شده‌اند که پهنای باند موجود را بین کاربران تقسیم کنند.

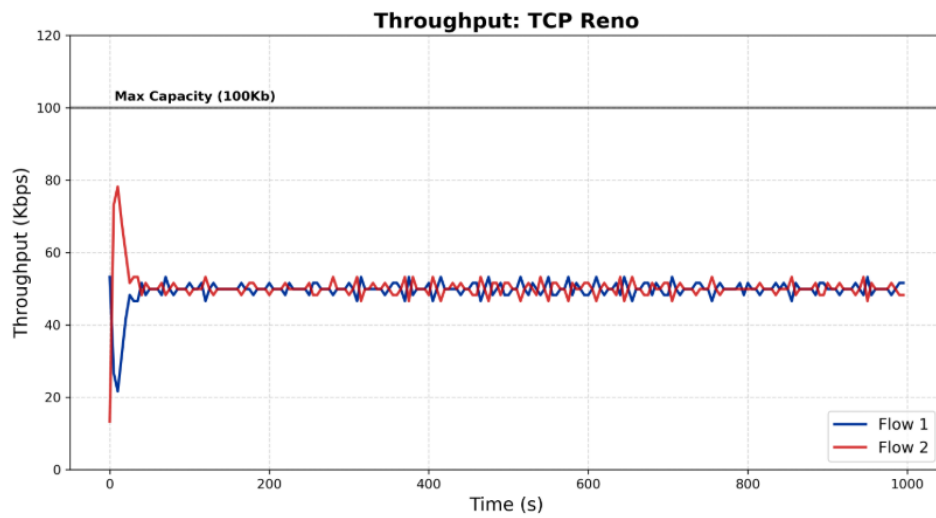
تفاوت‌ها: پروتکل Vegas معمولاً نوسان کمتری در گذردهی دارد، در حالی که Reno و Tahoe به دلیل مکانیزم‌های واکنشی (Reactive)، نمودارهای دندان‌اره‌ای (Sawtooth) شدیدتری دارند.



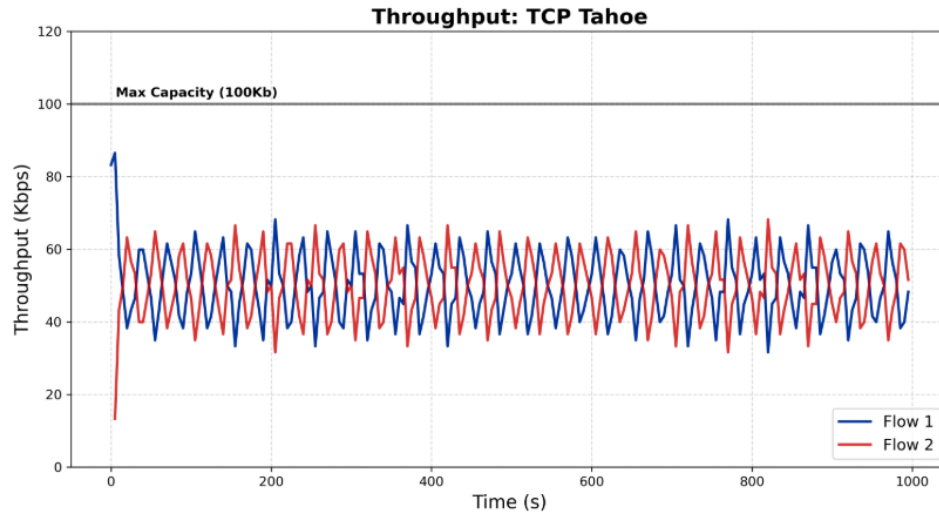
شکل ۲: مقایسه گذردهی پروتکل‌ها



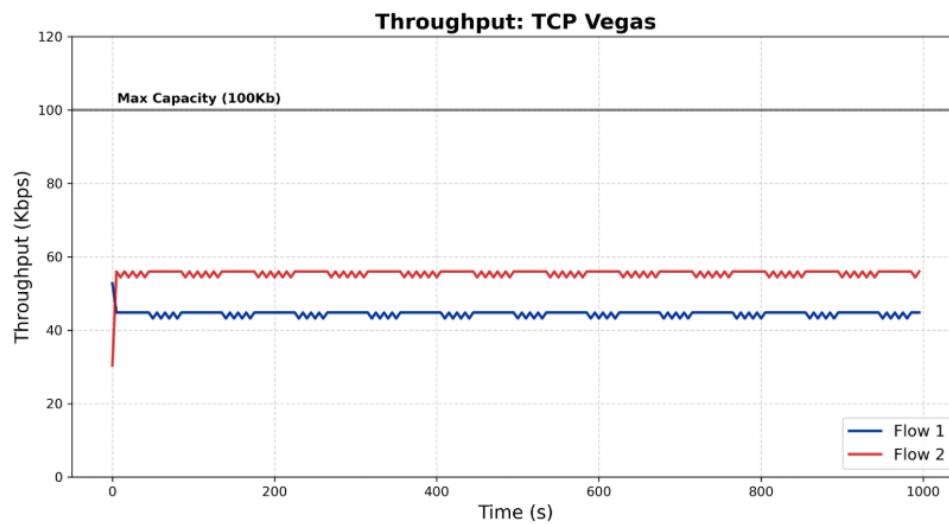
شکل ۳: تغییرات گذردهی در طول زمان



شکل ۴: تغییرات گذردهی Reno در طول زمان



شکل ۵: تغییرات گذردهی Tahoe در طول زمان

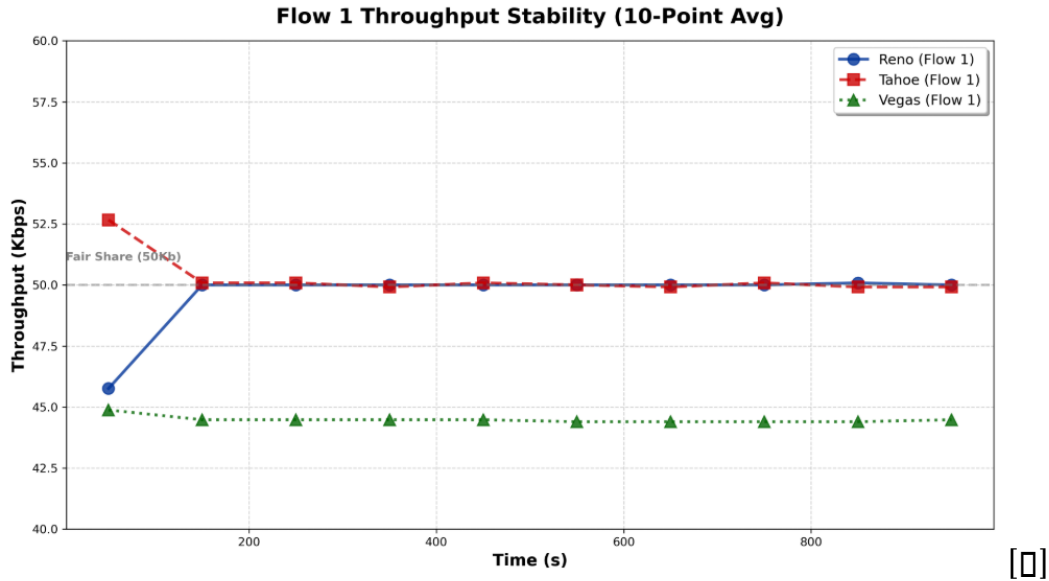


شکل ۶: تغییرات گذردهی Vegas در طول زمان

۵ مقایسه تاخیر رفت و برگشت

۱.۵ مشاهدات

میانگین RTT برای همه پروتکل‌ها بسیار بالا (حدود ۷۰۰ میلی‌ثانیه) است.



شکل ۷: تغییرات گذردهی در طول زمان

۲.۵ تحلیل تکنیکال

این پدیده به دلیل تاخیر صف (Queuing Delay) رخ می‌دهد:

۱. ظرفیت صف: ۱۰ بسته

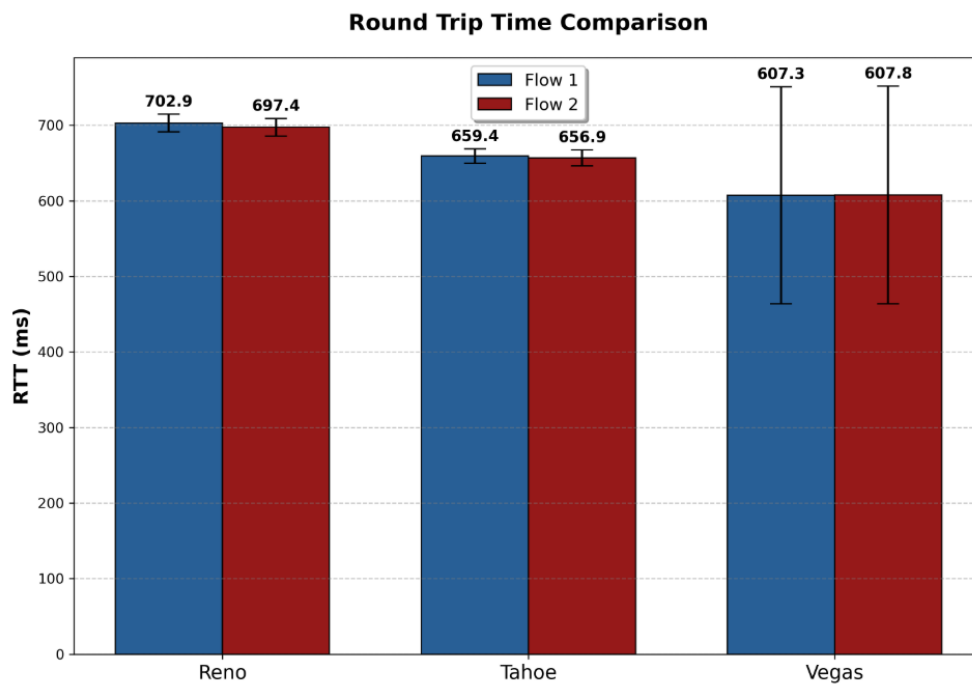
۲. حجم هر بسته: ۸۰۰۰ بیت

۳. سرعت لینک خروجی: ۱۰۰,۰۰۰ بیت بر ثانیه

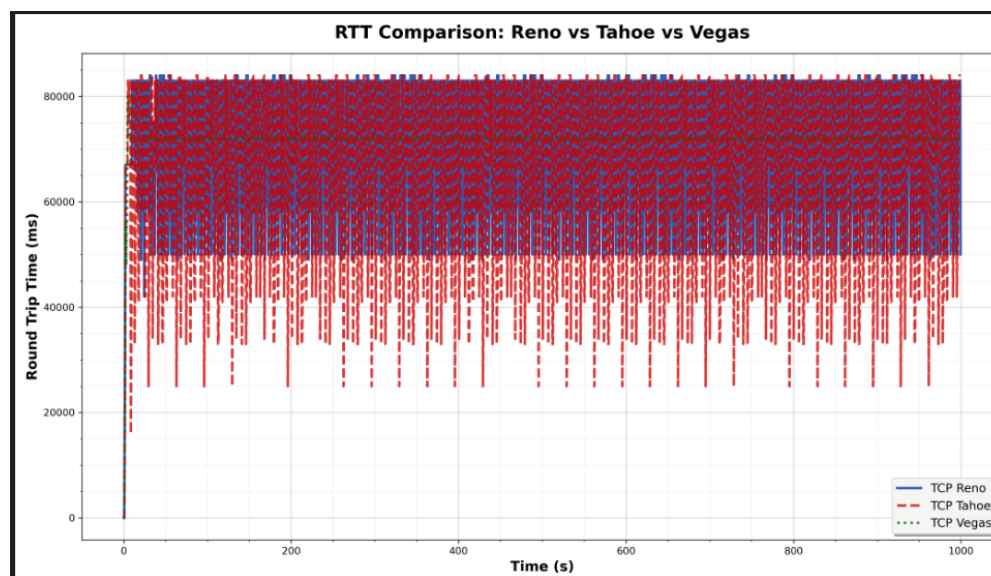
۴. زمان تخلیه صف پر:

$$\frac{10 \times 8000}{100000} = 0.8 \text{ s (800 ms)}$$

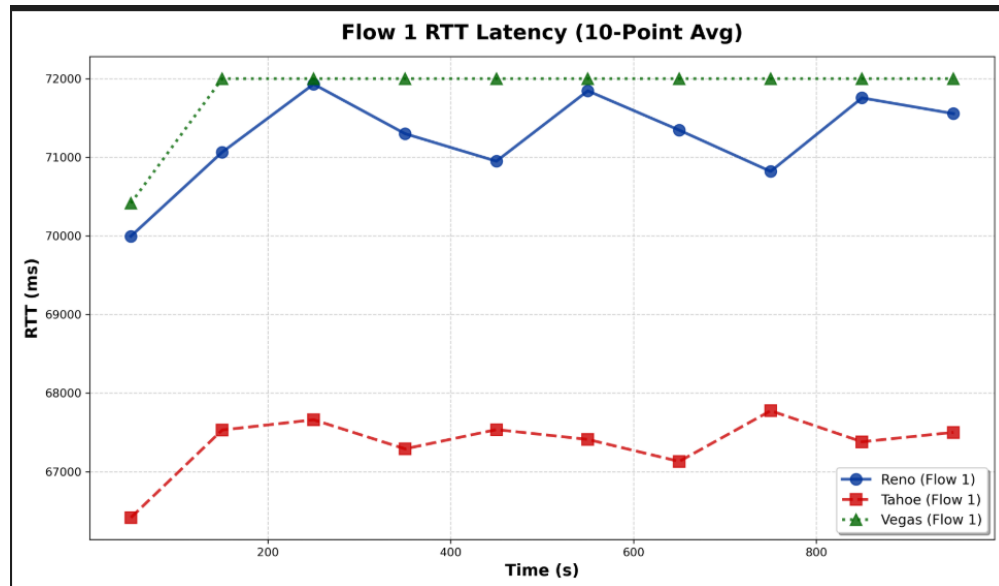
چون لینک اشباع است، صف همیشه پر است. بنابراین هر بسته باید حدود ۸۰۰ میلی‌ثانیه در صف منتظر بماند. این نشان می‌دهد که در این سناریو، گلوگاه اصلی بافر روتر است (پدیده Bufferbloat). پروتکل Vegas ممکن است RTT کمی پایین‌تر نشان دهد زیرا تلاش می‌کند قبل از پر شدن کامل صف، سرعت را کم کند.



شکل ۸: مقایسه تاخیر رفت و برگشت



شکل ۹: تغییرات تاخیر در طول زمان



شکل ۱۰: تغییرات تاخیر در طول زمان

۶ مقایسه نرخ از دست رفتن بسته

۱.۶ مشاهدات

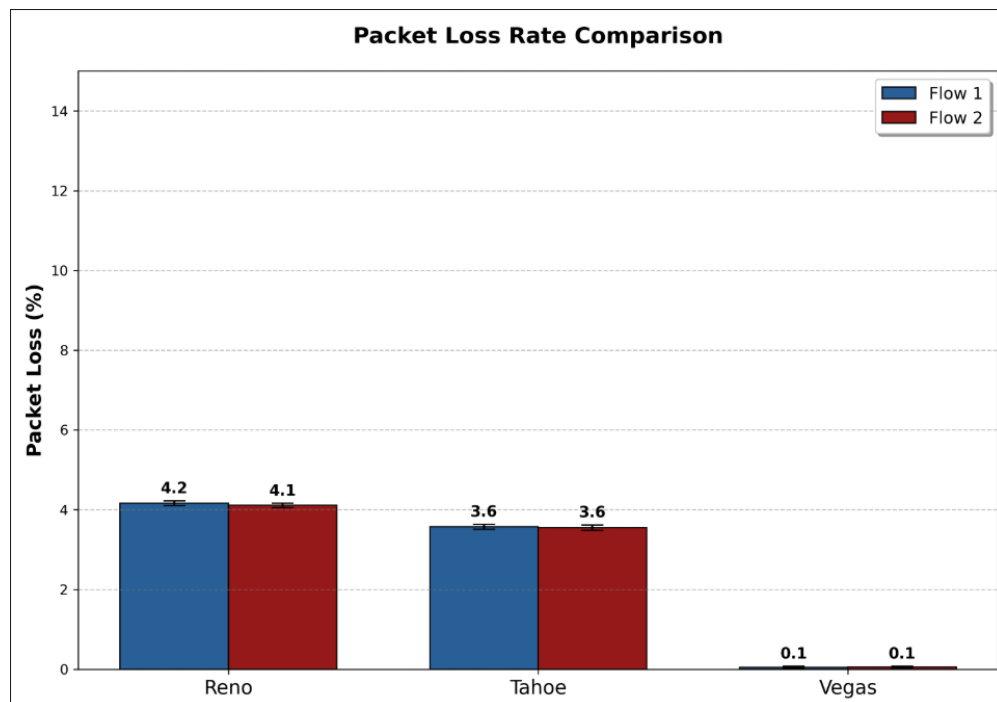
۱. Reno و Tahoe: دارای نرخ تلفات بالاتر (بین ۳ درصد تا ۵ درصد)

۲. Vegas: دارای نرخ تلفات بسیار پایین (نزدیک به ۰ درصد)

۲.۶ تحلیل مقایسه‌ای

TCP Reno/Tahoe (واکنشی): این پروتکل‌ها کور هستند. آن‌ها سرعت ارسال را افزایش می‌دهند تا زمانی که بافر روتر پر شود و یک بسته حذف شود (Drop). تنها در این لحظه است که می‌فهمند شبکه شلوغ است و سرعت را کم می‌کنند. بنابراین، Packet Loss جزوی از مکانیزم کاری آنهاست.

TCP Vegas (پیشگیرانه): این پروتکل هوشمندتر است. Vegas دائماً RTT را اندازه‌گیری می‌کند. اگر RTT شروع به افزایش کند (یعنی صف در حال پر شدن است)، Vegas قبل از اینکه بافر سرریز شود و بسته‌ای گم شود، سرعت خود را کاهش می‌دهد. به همین دلیل نرخ تلفات در Vegas بسیار ناچیز است.



شکل ۱۱: مقایسه نرخ از دست رفتن بسته

این پروژه با موفقیت تفاوت‌های بنیادین بین الگوریتم‌های کنترل ازدحام را در محیط NS2 شبیه‌سازی کرد. نتایج کلیدی عبارتند از:

۱. **پایداری:** هر سه پروتکل توانستند در شرایط بحرانی (پهنای باند ۱۰۰ کیلوبیت) پایداری شبکه را حفظ کنند

۲. **عدالت:** پهنای باند به طور مساوی بین جریان‌ها تقسیم شد

۳. **انتخاب پروتکل:**

(آ) اگر اولویت حداکثر استفاده از پهنای باند است و تلفات بسته اهمیت کمی دارد، Reno گزینه استاندارد و مناسبی است

(ب) اگر اولویت کاهش تلفات بسته و تاخیر کمتر است، Vegas عملکرد بسیار بهتری دارد، هرچند ممکن است در رقابت با پروتکل‌های تهاجمی مثل Reno، پهنای باند کمتری نصیبش شود

فایل‌های زیر به همراه این گزارش ارسال شده‌اند:

۱. کدهای TCL شبیه‌سازی (project_sim.tcl)

۲. اسکریپت‌های تحلیل داده (run_project.sh و rtt.awk)

۳. اسکریپت‌های رسم نمودار پایتون

۴. داده‌های خام آماری (all_runs_raw.csv)

این بخش مراحل دقیق انجام پروژه، از آماده‌سازی محیط تا تولید نمودارهای نهایی را تشریح می‌کند.

۷ گام اول: آماده‌سازی محیط اجرا

۱. ترمینال لینوکس (Ubuntu) یا محیط WSL را باز کنید

۲. پیش‌نیازهای نرم‌افزاری را نصب کنید:

(آ) اطمینان حاصل کنید که NS2 نصب است (دستور ns باید کار کند)

(ب) اطمینان حاصل کنید که NAM برای نمایش گرافیکی نصب است

(ج) پایتون ۳ و کتابخانه Matplotlib را برای رسم نمودار نصب کنید:

```
sudo apt-get update
sudo apt-get install python3-matplotlib python3-numpy
```

۸ گام دوم: ایجاد فضای کاری

۱. یک پوشه جدید برای پروژه بسازید:

```
mkdir -p CA2_Project/results
cd CA2_Project
```

۲. پنج فایل اصلی پروژه را ایجاد و ذخیره کنید (جزئیات کدها در پیوست ب آمده است):

(آ) project_sim.tcl (موتور شبیه‌سازی)

(ب) run_project.sh (مدیریت خودکار اجرا)

(ج) rtt.awk (محاسبه گر تاخیر)

(د) plot_timelines.py (رسم نمودارهای زمانی)

(ه) plot_professional.py (تحلیل آماری)

۹ گام سوم: مدیریت دسترسی‌ها

برای اینکه اسکریپت بتواند اجرا شود، باید مجوز اجرایی (Executable Permission) به آن بدهید:

```
chmod +x run_project.sh
```

۱۰ گام چهارم: اجرای آزمایش

اسکریپت اصلی را اجرا کنید. این دستور به تنهایی کل فرآیند آزمایش را انجام می‌دهد:

```
./run_project.sh
```

چه اتفاقی در پس‌زمینه رخ می‌دهد:

۱. اسکریپت ۳۰ بار اجرا می‌شود (۱۰ بار برای هر پروتکل: Reno, Tahoe, Vegas)

۲. برای هر اجرا، تاخیرها و زمان‌های شروع به صورت کاملاً تصادفی (Random) تولید می‌شوند

۳. فایل‌های Trace پردازش شده و مقادیر گذردهی (Throughput)، اتلاف (Loss) و تاخیر (RTT) محاسبه می‌شوند

۴. نتایج دقیق در فایل results/all_runs_raw.csv ذخیره می‌شوند

```
ser@LAPTOP-4LG474AL: /mnt/d/computernetwork/ns2/ns-allinone-2.35/ca2/animations$ cd ..
ser@LAPTOP-4LG474AL: /mnt/d/computernetwork/ns2/ns-allinone-2.35/ca2$ ./run_project.sh
[INIT] Cleaning old files...
=====
STARTING SIMULATION SUITE
Time: Mon Dec 22 17:07:20 +0330 2025
=====
[17:07:21] Processing Protocol: Reno
=====
[Run 1/10] Simulating... Done (2177ms). Parsing data...
-> Flow 1: Thr=49.5875 Kbps | Loss=4.2400 % | RTT=712.5508 ms
-> Flow 2: Thr=50.2448 Kbps | Loss=4.0500 % | RTT=688.2082 ms
[Info] Trace files saved for timeline plotting.
[Run 2/10] Simulating... Done (2070ms). Parsing data...
-> Flow 1: Thr=49.8454 Kbps | Loss=4.0900 % | RTT=712.0786 ms
-> Flow 2: Thr=50.0784 Kbps | Loss=4.1400 % | RTT=687.4255 ms
[Run 3/10] Simulating... Done (2008ms). Parsing data...
-> Flow 1: Thr=50.145 Kbps | Loss=4.1600 % | RTT=688.1029 ms
-> Flow 2: Thr=49.6624 Kbps | Loss=4.0600 % | RTT=712.6245 ms
[Run 4/10] Simulating... Done (2172ms). Parsing data...
-> Flow 1: Thr=49.9702 Kbps | Loss=4.2200 % | RTT=688.6796 ms
-> Flow 2: Thr=49.9203 Kbps | Loss=4.1000 % | RTT=710.6965 ms
[Run 5/10] Simulating... Done (1891ms). Parsing data...
-> Flow 1: Thr=49.6291 Kbps | Loss=4.0900 % | RTT=713.2956 ms
-> Flow 2: Thr=50.2614 Kbps | Loss=4.1700 % | RTT=686.7743 ms
[Run 6/10] Simulating... Done (1506ms). Parsing data...
-> Flow 1: Thr=50.2198 Kbps | Loss=4.1700 % | RTT=687.6868 ms
-> Flow 2: Thr=49.6125 Kbps | Loss=4.1100 % | RTT=713.3857 ms
[Run 7/10] Simulating... Done (1412ms). Parsing data...
-> Flow 1: Thr=49.6208 Kbps | Loss=4.1100 % | RTT=713.3472 ms
-> Flow 2: Thr=50.2365 Kbps | Loss=4.1400 % | RTT=688.1962 ms
[Run 8/10] Simulating... Done (1912ms). Parsing data...
-> Flow 1: Thr=49.8205 Kbps | Loss=4.2200 % | RTT=689.6025 ms
-> Flow 2: Thr=50.095 Kbps | Loss=4.0700 % | RTT=709.9299 ms
[Run 9/10] Simulating... Done (1583ms). Parsing data...
-> Flow 1: Thr=49.9619 Kbps | Loss=4.1000 % | RTT=710.9563 ms
-> Flow 2: Thr=49.9786 Kbps | Loss=4.2200 % | RTT=688.5399 ms
[Run 10/10] Simulating... Done (2116ms). Parsing data...
-> Flow 1: Thr=49.5875 Kbps | Loss=4.2400 % | RTT=712.5580 ms
-> Flow 2: Thr=50.2448 Kbps | Loss=4.0500 % | RTT=688.2085 ms
[Stats] Calculating averages for Reno...
[FINAL] Reno Results:
Flow 1 Avg: 49.83 Kbps, 4.16 %, 702.88 ms
Flow 2 Avg: 50.03 Kbps, 4.11 %, 697.39 ms
```

```

-----
[17:18:50] Processing Protocol: Tahoe
-----
[Run 1/10] Simulating... Done (2044ms). Parsing data...
-> Flow 1: Thr=50.2698 Kbps | Loss=3.5400 % | RTT=660.6327 ms
-> Flow 2: Thr=49.6458 Kbps | Loss=3.4700 % | RTT=661.3558 ms
[Info] Trace files saved for timeline plotting.
[Run 2/10] Simulating... Done (1905ms). Parsing data...
-> Flow 1: Thr=49.8371 Kbps | Loss=3.5700 % | RTT=668.6329 ms
-> Flow 2: Thr=50.0867 Kbps | Loss=3.6100 % | RTT=641.1036 ms
[Run 3/10] Simulating... Done (2223ms). Parsing data...
-> Flow 1: Thr=50.2531 Kbps | Loss=3.6600 % | RTT=640.8793 ms
-> Flow 2: Thr=49.6624 Kbps | Loss=3.5300 % | RTT=668.9529 ms
[Run 4/10] Simulating... Done (1460ms). Parsing data...
-> Flow 1: Thr=50.0784 Kbps | Loss=3.6600 % | RTT=668.6384 ms
-> Flow 2: Thr=49.8371 Kbps | Loss=3.5500 % | RTT=641.7075 ms
[Run 5/10] Simulating... Done (1490ms). Parsing data...
-> Flow 1: Thr=49.8621 Kbps | Loss=3.4700 % | RTT=661.0368 ms
-> Flow 2: Thr=50.1117 Kbps | Loss=3.6300 % | RTT=661.5745 ms
[Run 6/10] Simulating... Done (-323ms). Parsing data...
-> Flow 1: Thr=50.095 Kbps | Loss=3.6100 % | RTT=661.5529 ms
-> Flow 2: Thr=49.8288 Kbps | Loss=3.4600 % | RTT=661.2890 ms
[Run 7/10] Simulating... Done (2019ms). Parsing data...
-> Flow 1: Thr=49.8454 Kbps | Loss=3.4900 % | RTT=661.3102 ms
-> Flow 2: Thr=50.1034 Kbps | Loss=3.6100 % | RTT=661.6874 ms
[Run 8/10] Simulating... Done (1866ms). Parsing data...
-> Flow 1: Thr=50.1949 Kbps | Loss=3.6500 % | RTT=642.1258 ms
-> Flow 2: Thr=49.729 Kbps | Loss=3.5300 % | RTT=668.9454 ms
[Run 9/10] Simulating... Done (1809ms). Parsing data...
-> Flow 1: Thr=49.7789 Kbps | Loss=3.5300 % | RTT=668.6366 ms
-> Flow 2: Thr=50.1616 Kbps | Loss=3.6500 % | RTT=641.2577 ms
[Run 10/10] Simulating... Done (2217ms). Parsing data...
-> Flow 1: Thr=50.2698 Kbps | Loss=3.5400 % | RTT=660.6327 ms
-> Flow 2: Thr=49.6458 Kbps | Loss=3.4700 % | RTT=661.3359 ms
[Stats] Calculating averages for Tahoe...
[FINAL] Tahoe Results:
Flow 1 Avg: 50.04 Kbps, 3.57 %, 659.40 ms
Flow 2 Avg: 49.88 Kbps, 3.55 %, 656.92 ms

```

```

-----
[17:30:14] Processing Protocol: Vegas
-----
[Run 1/10] Simulating... Done (1429ms). Parsing data...
-> Flow 1: Thr=44.488 Kbps | Loss=.0700 % | RTT=718.6274 ms
-> Flow 2: Thr=55.432 Kbps | Loss=.0500 % | RTT=719.8281 ms
[Info] Trace files saved for timeline plotting.
[Run 2/10] Simulating... Done (2139ms). Parsing data...
-> Flow 1: Thr=33.312 Kbps | Loss=.0400 % | RTT=479.9742 ms
-> Flow 2: Thr=66.616 Kbps | Loss=.0400 % | RTT=480.0804 ms
[Run 3/10] Simulating... Done (1370ms). Parsing data...
-> Flow 1: Thr=44.52 Kbps | Loss=.0700 % | RTT=718.3409 ms
-> Flow 2: Thr=55.4 Kbps | Loss=.0700 % | RTT=719.8524 ms
[Run 4/10] Simulating... Done (2203ms). Parsing data...
-> Flow 1: Thr=66.608 Kbps | Loss=.0400 % | RTT=480.0826 ms
-> Flow 2: Thr=33.312 Kbps | Loss=.0400 % | RTT=479.9698 ms
[Run 5/10] Simulating... Done (1606ms). Parsing data...
-> Flow 1: Thr=55.464 Kbps | Loss=.0500 % | RTT=719.8643 ms
-> Flow 2: Thr=44.504 Kbps | Loss=.0700 % | RTT=718.8429 ms
[Run 6/10] Simulating... Done (1802ms). Parsing data...
-> Flow 1: Thr=44.488 Kbps | Loss=.0700 % | RTT=718.9007 ms
-> Flow 2: Thr=55.432 Kbps | Loss=.0800 % | RTT=719.9179 ms
[Run 7/10] Simulating... Done (1438ms). Parsing data...
-> Flow 1: Thr=33.296 Kbps | Loss=.0400 % | RTT=480.2363 ms
-> Flow 2: Thr=66.648 Kbps | Loss=.0400 % | RTT=480.0079 ms
[Run 8/10] Simulating... Done (2006ms). Parsing data...
-> Flow 1: Thr=44.48 Kbps | Loss=.0700 % | RTT=718.8142 ms
-> Flow 2: Thr=55.44 Kbps | Loss=.0700 % | RTT=719.8730 ms
[Run 9/10] Simulating... Done (1455ms). Parsing data...
-> Flow 1: Thr=49.96 Kbps | Loss=.0100 % | RTT=319.9592 ms
-> Flow 2: Thr=49.984 Kbps | Loss=.0300 % | RTT=319.9295 ms
[Run 10/10] Simulating... Done (1652ms). Parsing data...
-> Flow 1: Thr=44.488 Kbps | Loss=.0700 % | RTT=718.6274 ms
-> Flow 2: Thr=55.432 Kbps | Loss=.0500 % | RTT=719.8281 ms
[Stats] Calculating averages for Vegas...
[FINAL] Vegas Results:
Flow 1 Avg: 46.11 Kbps, .05 %, 607.34 ms
Flow 2 Avg: 53.82 Kbps, .05 %, 607.81 ms

```

```

=====
COMPLETE! Data saved to results/all_runs_raw.csv
=====

```

۱۱ گام پنجم: بصری‌سازی و گزارش‌دهی

پس از اتمام شبیه‌سازی، دستورات زیر را برای تولید نمودارها اجرا کنید:

```
python3 plot_timelines.py
```

```
python3 plot_professional.py
```

خروجی: به پوشه‌های graphs_timeline و graphs_professional مراجعه کنید و تصاویر تولید شده را در گزارش قرار دهید.

پیوست ب: تحلیل خط به خط کدها

۱۲ اسکریپت شبیه‌سازی: project_sim.tcl

این اسکریپت فیزیک شبکه و توپولوژی را تعریف می‌کند.

```
if {$argc != 2} { ... }
set protocol [lindex $argv 0]
set seed [lindex $argv 1]

set ns [new Simulator]

set rng_delay [new RNG]
$rng_delay seed $seed
set rand_delay [new RandomVariable/Uniform]
$rand_delay set min_ 5.0
$rand_delay set max_ 25.0

set tf [open "out.tr" w]
$ns trace-all $tf

set n0 [$ns node]
set n2 [$ns node]

$ns duplex-link $n2 $n3 100Kb 1ms DropTail
$ns queue-limit $n2 $n3 10

proc attach-tcp-agent {node variant fid} {
    global ns
```

```
if {$variant == "Reno"} {  
    set tcp [new Agent/TCP/Reno]  
} elseif {$variant == "Tahoe"} {  
    set tcp [new Agent/TCP]  
} elseif {$variant == "Vegas"} {  
    set tcp [new Agent/TCP/Vegas]  
}  
  
$tcp set fid_ $fid  
$tcp set packetSize_ 1000  
$ns attach-agent $node $tcp  
return $tcp  
}  
  
$ns at 1000.0 "finish"  
$ns run
```

تحلیل بخش‌های کلیدی:

۱. بررسی می‌کند که کاربر پروتکل و Seed را وارد کرده باشد

۲. آرگومان اول (نام پروتکل) را می‌خواند

۳. آرگومان دوم (عدد تصادفی Seed) را می‌خواند

۴. ایجاد نمونه اصلی شبیه‌ساز NS2

۵. ایجاد تولیدکننده عدد تصادفی برای تاخیر لینک

۶. تنظیم Seed برای تکرارپذیری آزمایش

۷. تنظیم توزیع یکنواخت (Uniform)

۸. حداقل تاخیر ۵ میلی‌ثانیه

۹. حداکثر تاخیر ۲۵ میلی‌ثانیه

۱۰. باز کردن فایل برای نوشتن رخدادها

۱۱. دستور به NS2 برای ثبت تمام رخدادها در این فایل

۱۲. ایجاد گره ۰ (فرستنده ۱)

۱۳. ایجاد گره ۲ (روتر ۱)

۱۴. لینک $n2 \rightarrow n3$: پهنای باند 100Kbps، تاخیر 1ms، صف DropTail

۱۵. تنظیم دقیق ظرفیت صف به ۱۰ بسته

۱۶. انتخاب کلاس C++ داخلی بر اساس نام پروتکل

۱۷. شناسه جریان (برای رنگ‌بندی: ۱ = آبی، ۲ = قرمز)

۱۸. تنظیم اندازه بسته به ۱۰۰۰ بایت

۱۹. اتصال ایجنت TCP به گره

۲۰. تنظیم زمان پایان شبیه‌سازی در ۱۰۰۰ ثانیه

۲۱. شروع ساعت شبیه‌سازی

۱۳ اسکریپت مدیریت خودکار: run_project.sh

این اسکریپت وظیفه اجرای دسته‌ای (Batch) آزمایش‌ها را بر عهده دارد.

```
#!/bin/bash

rm -rf traces results
mkdir -p traces results

echo "Protocol,Flow,RunID,Thr,RTT,Loss" > results/all_runs_raw.csv

protocols=("Reno" "Tahoe" "Vegas")
for proto in "${protocols[@]}; do

    for i in {1..10}; do

        ns project_sim.tcl $proto $i > /dev/null 2>&1

        trace="out_${proto}_${i}.tr"

        thr1=$(awk ... $trace)

        s1=$(awk ... $trace)
```

```
r1=$(awk ... $trace)

rtt1=$(awk -f rtt.awk ... $trace)

echo "$proto,Flow1,$i,$thr1,$rtt1,$l1" >> results/all_runs_raw.csv

done

avg_t1=$(echo "scale=2; $sum_thr1/10"|bc)

echo "$proto,Flow1,AVERAGE,$avg_t1..." >> results/all_runs_raw.csv

done
```

تحلیل بخش‌های کلیدی:

۱. حذف فایل‌های قدیمی برای جلوگیری از تداخل
۲. ایجاد پوشه‌های جدید
۳. ایجاد هدر (سرستون) برای فایل CSV نهایی
۴. تکرار برای هر پروتکل
۵. تکرار ۱۰ بار (اجرای ۱ تا ۱۰)
۶. فراخوانی NS2 با پروتکل خاص و Seed مشخص
۷. دستور awk فایل trace را اسکن می‌کند
۸. شمارش ارسال در مبدا
۹. شمارش دریافت در مقصد
۱۰. فراخوانی اسکریپت خارجی rtt.awk برای محاسبه تاخیر
۱۱. افزودن مقادیر محاسبه شده به فایل CSV
۱۲. استفاده از bc (ماشین حساب لینوکس) برای محاسبه میانگین ۱۰ اجرا
۱۳. ذخیره میانگین به عنوان یک سطر ویژه در فایل CSV

۱۴ محاسبه‌گر تاخیر: rtt.awk

این اسکریپت فاصله زمانی بین ارسال بسته و دریافت تاییدیه (ACK) آن را محاسبه می‌کند.

```
BEGIN { count = 0; sum_rtt = 0; }

{
    event = $1; time = $2; node = $3; seq_num = $11;

    if (event == "+" && node == fsrc && $5 == "tcp") {
        send_time[seq_num] = time;
    }

    if (event == "r" && $4 == fsrc && $5 == "ack") {
        if (seq_num in send_time) {
            rtt = time - send_time[seq_num];
            sum_rtt += rtt;
            count++;
            delete send_time[seq_num];
        }
    }
}

END { printf "%.4f", (sum_rtt / count) * 1000; }
```

تحلیل بخش‌های کلیدی:

۱. مقداردهی اولیه متغیرها
۲. اگر بسته از مبدا ارسال شود و از نوع TCP باشد
۳. زمان ارسال را با کلید شماره ترتیب ذخیره می‌کند
۴. اگر بسته در مبدا دریافت شود و از نوع ACK باشد
۵. بررسی می‌کند که آیا زمان ارسال برای این شماره ترتیب وجود دارد
۶. محاسبه تاخیر (زمان حال - زمان ارسال)
۷. افزودن به مجموع
۸. پاکسازی حافظه
۹. محاسبه نهایی: میانگین RTT به میلی ثانیه

۱۵ بصری‌سازی: plot_timelines.py

این اسکریپت نمودارهای خطی با کیفیت بالا (Ultra-HD) تولید می‌کند.

```
import matplotlib.pyplot as plt

def get_thr_data(proto):
    with open(filename, 'r') as f:
        for line in f:
            if 'r' in line:
                idx = int(time / INTERVAL)
                buckets[idx] += size

    throughput = (buckets[idx] * 8) / (1000.0 * INTERVAL)
    return throughput

fig, ax = plt.subplots(figsize=(18, 10))

ax.plot(time, throughput, label="Reno Flow 1", color='#003399')

ax.axhline(100, color='black', linewidth=2)
ax.text(0, 102, 'MAX (100Kb)')

plt.savefig("timeline_throughput.png", dpi=300)
```

تحلیل بخش‌های کلیدی:

۱. خواندن فایل trace

۲. فقط بسته‌های دریافتی

۳. منطق دسته‌بندی (Bucketing)

۴. زمان را بر بازه (مثلاً ۱۰ ثانیه) تقسیم می‌کند تا بسته‌ها گروه‌بندی شوند

۵. تبدیل واحد: $\text{Kbps} = (\text{Bytes} * 8) / (1000 * \text{Interval_Seconds})$

۶. تنظیم ابعاد تصویر (HD)

۷. رسم خطوط

۸. رسم یک خط افقی سیاه در $y = 100$ (ظرفیت گلوگاه)

۹. برچسب خط

۱۰. ذخیره فایل با کیفیت ۳۰۰ نقطه در اینچ

Tcl

```

# --- Inputs ---
if {$argc != 2} { ... }      # Checks if user provided Protocol and Seed arguments
set protocol [lindex $argv 0] # Reads the first argument (Protocol name).
set seed [lindex $argv 1]    # Reads the second argument (Random Seed).
set ns [new Simulator]       # Creates the main NS2 Simulator instance.

# --- Randomization (Jitter and Delay) ---
set rng_delay [new RNG]      # Creates a Random Number Generator for link delay
$rng_delay seed $seed        # Sets the Seed for experiment reproducibility.
set rand_delay [new RandomVariable/Uniform] # Sets Uniform distribution.
$rand_delay set min_ 5.0     # Min delay 5 ms.
$rand_delay set max_ 25.0    # Max delay 25 ms.

# (Same logic repeats for Start Time to prevent flow synchronization)

# --- Tracing ---
set tf [open "out.tr" w]     # Opens file for writing events.
$ns trace-all $tf           # Tells NS2 to log ALL events to this file.

# --- Topology Definition ---
set n0 [$ns node]            # Creates Node 0 (Sender 1).
set n2 [$ns node]            # Creates Node 2 (Router 1).
# ... (Repeats for other nodes) ...

# Link Configuration [Bottleneck Definition]
# Link n2->n3: 100Kbps Bandwidth, 1ms Delay, DropTail Queue
$ns duplex-link $n2 $n3 100Kb 1ms DropTail
$ns queue-limit $n2 $n3 10   # Sets exact queue capacity to 10 packets.

# --- Protocol Settings (Agent Setup) ---
proc attach-tcp-agent {node variant fid} {
    global ns
    # Selects internal C++ class based on protocol name
    if {$variant == "Reno"} { set tcp [new Agent/TCP/Reno] }
    elseif {$variant == "Tahoe"} { set tcp [new Agent/TCP] }
    elseif {$variant == "Vegas"} { set tcp [new Agent/TCP/Vegas] }

    $tcp set fid_ $fid        # Flow ID (for coloring: 1=Blue, 2=Red).
    $tcp set packetSize_ 1000 # Sets packet size to 1000 bytes.
    $ns attach-agent $node $tcp # Attaches TCP agent to the node.
    return $tcp
}

# --- Execution ---
$ns at 1000.0 "finish"       # Sets simulation end time to 1000 seconds.
$ns run                      # Starts the simulation clock.

```

```
Bash

#!/bin/bash

# --- Setup ---
rm -rf traces results          # Deletes old files to prevent conflict.
mkdir -p traces results        # Creates new directories.

# Creates header for the final CSV file.
echo "Protocol,Flow,RunID,Thr,RTT,Loss" > results/all_runs_raw.csv

# --- Main Loop ---
protocols=("Reno" "Tahoe" "Vegas")
for proto in "${protocols[@]}; do    # Iterates for each protocol.

    for i in {1..10}; do            # Iterates 10 times (Run 1 to 10).

        # Run Simulation
        # Calls NS2 with specific Protocol and Seed ($i)
        ns project_sim.tcl $proto $i > /dev/null 2>&1

        trace="out_${proto}_${i}.tr"

        # --- Analysis with AWK (Mathematical Calculations) ---

        # 1. Calculate Throughput
        # awk scans the trace file.
        # $1=="r": Event is Receive.
        # $4==dest: Packet reached destination.
        # {sum+=$6}: Sums up packet size.
        # Convert bits to Kilobits/Megabits.
        thr1=$(awk ... $trace)

        # 2. Calculate Packet Loss
        # Count Sent (+): At source.
        # Count Received (r): At destination.
        # Formula: (Sent - Received) / Sent.
        s1=$(awk ... $trace)
        r1=$(awk ... $trace)

        # 3. Calculate RTT
        # Calls external script rtt.awk to calculate delay.
        rtt1=$(awk -f rtt.awk ... $trace)

        # --- Logging ---
        # Appends calculated values to CSV file.
        echo "$proto,Flow1,$i,$thr1,$rtt1,$l1" >> results/all_runs_raw.csv

    done

done

# --- Averaging ---
# Uses 'bc' (Linux calculator) to calculate average of 10 runs.
avg_t1=$(echo "scale=2; $sum_thr1/10"|bc)
```

```

Awk

BEGIN { count = 0; sum_rtt = 0; } # Initialize variables

{
    event = $1; time = $2; node = $3; seq_num = $11;

    # If packet is Sent (+) from source and is type TCP:
    if (event == "+" && node == fsrc && $5 == "tcp") {
        send_time[seq_num] = time; # Store send time keyed by "Sequence Number".
    }

    # If packet is Received (r) at source and is type ACK:
    if (event == "r" && $4 == fsrc && $5 == "ack") {
        # Check if send time exists for this sequence number?
        if (seq_num in send_time) {
            rtt = time - send_time[seq_num]; # Calculate delay (Current Time - Se
            sum_rtt += rtt;                  # Add to sum
            count++;
            delete send_time[seq_num];      # Clear memory
        }
    }
}

# Final Calculation: Average RTT in ms
END { printf "%.4f", (sum_rtt / count) * 1000; }

```

[۷] [۶] [۵] [۴] [۳] [۲] [۱]

مراجع

- [1] "Computer networks," https://gaia.cs.umass.edu/kurose_oss/ninth.php, 2024.
- [2] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR Congestion Control," Internet-Draft draft-cardwell-icrg-bbr-congestion-control-01, Internet Engineering Task Force (IETF), 2017. :
- [3] N. Cardwell, Y. Cheng, and V. Jacobson, "BBR Congestion Control: IETF 97 Presentation Slides," <https://www.ietf.org/proceedings/97/slides/slides-97-icrg-bbr-congestion-control-02.pdf>, 2016. IETF 97 :

-
- [4] Slogix, “NS2 Simulation Samples and Projects,” <https://slogix.in/ns2-simulation-samples>, 2024. : .
- [5] Loyola University Chicago, “An Introduction to Computer Networks: NS2 Tutorial,” <http://intronetworks.cs.luc.edu/current/html/ns2.html>, 2024. : .
- [6] Internet Society, “TCP Congestion Control Proceedings,” in *INET 2000 Conference Proceedings*, 2000. : .
- [7] xAI, “Grok assistance for latex formatting and troubleshooting,” <https://x.ai/grok>, dec 2025. Accessed: 2025-12-20.